

A Survey on Large Language Models for Personalized and Explainable Recommendations

Junyi CHEN

December 2023

Abstract

In recent years, Recommender Systems(RS) have witnessed a transformative shift with the advent of Large Language Models(LLMs) in the field of Natural Language Processing(NLP). These models such as OpenAI's GPT-3.5/4, Llama from Meta, have demonstrated unprecedented capabilities in understanding and generating human-like text. This has led to a paradigm shift in the realm of personalized and explainable recommendations, as LLMs offer a versatile toolset for processing vast amounts of textual data to enhance user experiences. To provide a comprehensive understanding of the existing LLM-based recommendation systems, this survey aims to analyze how RS can benefit from LLM-based methodologies. Furthermore, we describe major challenges in Personalized Explanation Generating(PEG) tasks, which are cold-start problems, unfairness and bias problems in RS.

1 Introduction

Recommendation systems are pivotal in aiding users to discover pertinent and personalized items or content. The advent of Large Language Models (LLMs) in Natural Language Processing (NLP) has sparked increased enthusiasm for leveraging the capabilities of these models to elevate and improve recommendation systems.

With the thriving of pre-training in NLP, many language models have been pre-trained on large scale unsupervised corpora, and then fine-tuned for down-

stream tasks. The transformer architecture[1], was introduced in 2017, has become a foundation in LLMs. It eschewed the sequential nature of recurrent neural networks (RNNs) in favor of a self-attention mechanism, enabling parallelization and significantly improving efficiency in handling sequential data. Based on Transformer architecture, many Pre-trained Language Models(PLMs) have emerged. GPT series[2], developed by OpenAI, and BERT[3], developed by Google, represent two prominent approaches to leveraging transformers for PLMs. The key advantage of incorporating PLMs into recommendation systems lies in their ability to extract high-quality representations of textual features and leverage the extensive external knowledge encoded within them.[4] Different from traditional recommendation systems, the LLM-based models can capture contextual information, comprehending user queries, item descriptions, and other textual data more efficiently.[5] Based on PLMs, fine-tuning strategy involves training the model on a smaller task-specific dataset. This dataset is typically related to a specific application or domain, such as sentiment analysis, text classification, question answering, or recommender systems. However, fine-tuning large language models on specific downstream tasks paradigm usually needs to fine-tunes all of the parameters in a PLM, which is a computational resource consuming process. Most researchers and companies cannot access as much resource as OpenAI or Microsoft or Google. As a result, a recently proposed paradigm, prompt learning[6], further unifies the use of PLMs on different tasks in a simple yet flexible manner. In general, prompting is the pro-

cess of providing additional information for a trained model to condition while predicting output labels for a task, for example, providing a piece of text inserted in the input examples. Prompt tuning is different from fine tuning in that it only requires storing a small task-specific prompt for each task instead of making a task specific copy of the entire PLM in fine tuning. The advantage of this paradigm lie in two aspects: (1) It bridges the gap between pre-training and downstream objectives, allowing better utilization of the rich knowledge in pretrained models. This advantage will be multiplied when very little downstream data is available. (2) Only a small set of parameters are needed to tune for prompt engineering, which is more efficient.[7]

This survey focuses on utilizing LLMs for Personalized Explanation Generating task. I will discuss the background of NLP for text generation in Chapter 2, the challenges existed in explainable recommendations in Chapter 3, the current approaches intending to solve these challenges in Chapter 4, and finally, the conclusion in Chapter 5.

2 Natural Language Generation

Natural Language Generation(NLG) stands at the forefront of artificial intelligence, representing a transformative field that empowers machines to comprehend and generate human-like text. NLG plays a pivotal role, bridging the gap between machines and humans through the creation of coherent, contextually relevant language. Especially, in the recommendation field, NLG approaches have a great potential to be applied to generating justifications for recommending results, which enhances the persuasiveness and effectiveness of Recommender systems.

2.1 Methodologies

The early stages of NLG were marked by rule-based approaches, where explicit linguistic rules were crafted to govern the generation of text. These systems operated on predefined templates and grammatical structures, limiting their flexibility and adapt-

ability to diverse applications. A significant leap came with the advent of neural network architectures, particularly recurrent neural networks (RNNs)[8]. RNNs allowed for sequential processing, enabling models to better capture contextual dependencies in language. However, they faced challenges with long-term dependencies, limiting their effectiveness in handling complex language structures. The turning point in NLG arrived with the introduction of the transformer architecture[1], the PLMs based on transformer architecture harnessed the power of attention mechanisms, enabling parallel processing and capturing intricate contextual relationships across words.

2.1.1 Transformer Architecture

In this section, we briefly go through Transformer architecture. Transformer consists of n identical layers, the l th layer encodes the previous layer's output S_{l-1} into $S_l \in \mathbb{R}^{|S| \times d}$, where $|s|$ is the length of the input token sequence, and d denotes the dimension of token embeddings. Each layer is composed of two sub-layers: Multi-Head Self-Attention(MHSA) and Feed-Forward Network. For self-attention mechanism, the input consists of queries and keys of dimension d_k , and values of dimension d_v . The attention scores can be calculated.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (1)$$

where Q, K, V are the query, key and value matrices obtained by linear transformations of the input sequence. d_k is the dimension of the key vectors. Multi-Head Attention is then calculated:

$$MultiHead(Q, K, V) = Concat(head_i, ..., head_h)W_O \quad (2)$$

where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$, the parameter matrices are denoted as: $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_k}$, and $W_O \in \mathbb{R}^{h_{dv} \times d_{model}}$. In addition to attention sub-layers, each of the layers in the encoder and decoder contains feed-forward network. The FFN section is a two-layer FFN with the ReLU activation function.

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2 \quad (3)$$

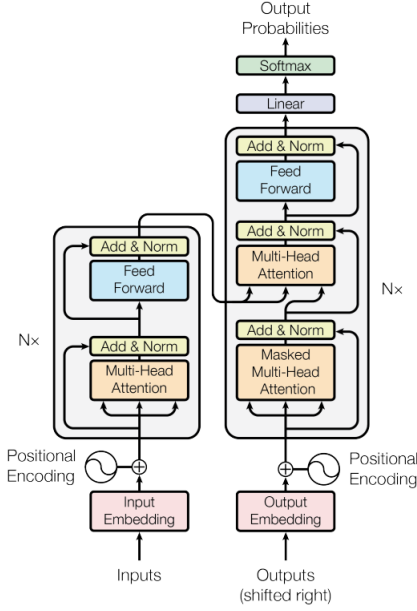


Figure 1: Architecture of Transformer

x is the MHSA's output, W_1, b_1, W_2, b_2 are weight parameters. We focus on Transformer-based pre-training language models. Bidirectional Encoder Representations from Transformer (BERT), Generative Pre-trained Transformer (GPT), and Text-to-Text Transfer Transformer (T5) are three main architectures. BERT is an encoder-only model, it is pretrained by predicting missing words in a bidirectional manner. While GPT is an example of decoder-only model, it predicts the next word in a sequence given its context. T5 is an encoder-decoder model, where both inputs and outputs are represented as textual sequences. Recent open-source PLM Llama-2[9], developed by Meta, ranges in scale from 7 billion to 70 billion parameters. And its fine-tuned LLMs, called Llama-2-chat, outperform open-source chat models on most benchmarks.

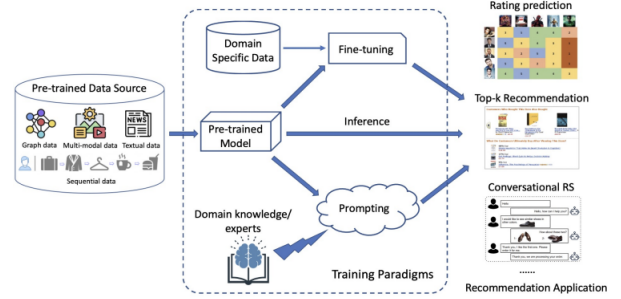


Figure 2: A generic architecture of language modeling paradigm for recommendation purpose

2.1.2 Training strategies

Given the significant impact that PLMs have achieved on NLP tasks in the pre-train and fine-tune paradigm, there has been a surge in adapting such paradigms to recommendation tasks. Figure 2 shows a high-level overview of the PLM for recommender systems.[7]

Many researchers adopt pre-train strategy. For example, BERT4Rec[10] models sequential user behavior with a bidirectional self-attention network through Cloze task, and Transformer4Rec[11] adopts a huggingface transformer-based architecture as the base model for next-item prediction. These two models have laid the foundation for LLM-based recommender systems.

Further more, some not only pre-train the model, but also fine-tune it with different specific datasets. Fine-tuning process will adjust the whole model parameters, for example, Petrov and Macdonald proposed GPTRec[12], which is a generative sequential recommendation model based on GPT-2. GPTRec is based on generative LLM, while BERT4Rec is based on discriminative LLM. It uses SVD tokenization for memory efficiency. UP5[13] is a P5-based model, it includes two sub-modules: a personalized prefix prompt that enhances fairness with respect to individual sensitive attributes and a prompt mixture that integrates multiple counterfactually-fair prompts for a set of sensitive attributes. For parameter-efficient training, they only optimize the

parameters in the prefix prompt and leave the PLM for RS untainted. Since tuning the whole model is usually time-consuming and less flexible, many choose to fine-tune only partial parameters of the model to balance between training overhead and recommendation performance.[14] This leads to Parameter-Efficient Fine-Tuning(PEFT) approaches. In order to reduce the memory consumption, there are two main lines applied in PEFT. One is to add small neural modules to the PLMs and fine-tune only these modules for each task. Adapter tuning inserts small neural modules between the layers of the base model. Prompt tuning attaches additional trainable tokens to the input or hidden layers. Another line is considering the incremental updates of the PLMs, without modifying the model architecture. Hu et al.(2022)[15]propose LoRA(**L**ow **R**ank **A**daptation), which parameterizes the update Δ as a low-rank matrix by the product of two small matrices:

$$W = W^{(0)} + \Delta = W^{(0)} + BA \quad (4)$$

During this fine-tuning, where $\Delta \in R^{d_1 \times d_2}$, we only need to update $A \in R^{r \times d_2}$ and $B \in R^{d_1 \times r}$. Because the rank r is much smaller than the dimension of W , the training overhead can be reduced up to 70% in total. This technique has significantly improved the implementing of affordable fine-tuning.

Instead of adapting PLMs to different downstream recommendation tasks by designing specific objective functions for fine-tuning, there is a rising trend to reformulate recommendation tasks through hard/soft prompts. Figure 3 [16]shows the detailed explanation of four different training manners for LLM-based recommendations. Generally speaking, prompting paradigm aims to design more suitable instructions and prompts to assist LLMs for better understanding of recommendation tasks such as rating predictions, sequential recommendation, direct recommendation, explanation generation, and review summarization. This prompting paradigm can be categorized into two training strategies, which are Tuning-free prompting and Tuning-required prompting. The former can be referred to as zero-shot recommendations in that this usually generate recommendations or/and subtasks without changing the parameters of PLMs. The latter training strategy either tunes the

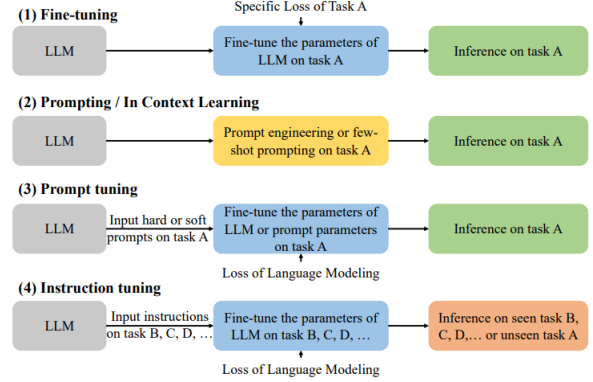


Figure 3: Four different training manners for LLM-based recommendations

parameters of PLMs or prompt-relevant parameters, or both of them. For example, Li et al.[17]proposes a continuous prompt learning approach by first fixing the PLM, tuning the prompt to bridge the gap between the continuous prompts and the loaded PLM, and then fine-tuning both the prompt and PLM, resulting in a higher BLUE score. I will discuss this approach in details in the next chapter.

3 Explanation Generating for Recommendation

3.1 PEPLER

PEPLER[17] is proposed for PEG task. It includes two methods which the author put them as discrete prompt learning and continuous prompt learning. In discrete(hard) prompt learning, a key idea is to represent the IDs in Recommender systems as domain-specific words such as movie titles and item features. This bridges the gap between Recommendation task and Natural language Generation task because IDs are indispensable in Recommender systems. Figure 4 shows a structure of PEPLER-D method, for each user-item pair (u, i) , the features in $F_u \cap F_i$ are more informative because they are both related to user and item. If the size is too small, $(F_u \cup F_i)/(F_u \cap F_i)$ can also be taken into considera-

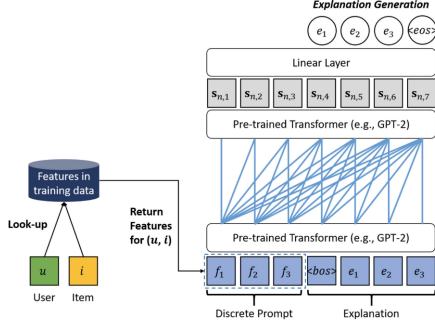


Figure 4: the PEPLER-D method that utilizes item features as a discrete prompt

tion. The input sequence to the pre-trained model is $S = [f_1, \dots, f_{|F_{u,i}|}, e_1, \dots, e_{|E_{u,i}|}]$, $|F_{u,i}|$ and $|E_{u,i}|$ denote the number of features and explanation words, respectively. we can obtain the sequence final representation $S_n = [s_{n,1}, \dots, s_{n,|S|}]$, a linear layer is applied to each token's final representation to map it onto a $|V|$ -sized vector. For example, $s_{n,t}$ becomes c_t after passing through this layer:

$$c_t = \text{softmax}(W^v s_{n,t} + b^v) \quad (5)$$

The vector c_t represents the probability distribution over the vocabulary V , for model learning loss function, Negative Log-likelihood(NLL) is applied.

$$\mathcal{L}_D = \frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} \frac{1}{|E_{u,i}|} \sum_{t=1}^{|E_{u,i}|} -\log c_{|F_{u,i}|+t}^{e_t} \quad (6)$$

where the probability c^{e_t} is offset by $|F_{u,i}|$ positions because the explanation is placed at the end of the sequence. Another method is continuous(or soft) prompt learning. Prompts do not necessarily have to be words or even readable, but can be vector representations. In this case, the input sequence can be represented as $S = [u, i, e_1, \dots, e_{|E_{u,i}|}]$, the user u 's vector representation can be retrieved via: $u = U^T g(u)$, $g(u) \in (0,1)^{|U|}$ denotes a one-hot vector, whose non-zero element corresponds to the position that user u 's vector locates in U . The vector i can be obtained in a similar way. The follow-up steps are identical to discrete prompt learning.

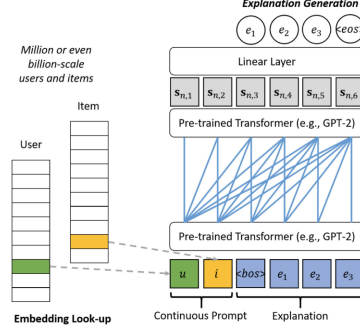


Figure 5: the PEPLER method that treats user and item IDs as continuous prompts

3.2 Rationale Generation and In-text Learning

Google research team introduces Chain of Thought(CoT)—a series of intermediate reasoning steps, which significantly improves the ability of LLM to perform complex reasoning. Providing or automatically generating rationale examples have proven to be effective. However, when tackling different downstream tasks, specific CoT usually need to be designed manually, this is not only time consuming and ineffective, but the performances also could vary tremendously. STaR[18] proposes to generate a new rationale by providing the model with the correct answer when each problem failure occurs. These generated rationales are then collected as part of the training data, which improves overall accuracy. Similar approach could be taken into consideration when solving PEG problems.

3.3 Evaluation Metrics

Explainability evaluation contains various aspects. For evaluating the quality of the generated explanations, there are different metrics which are widely accepted. BLEU[19] and ROUGE[20] are two widely used metrics in machine translation and text summarization respectively. BLEU stands for Bilingual evaluation understudy, it calculates the precision by comparing the n-grams(contiguous sequences of n items,

like words or characters) in the machine-generated translation with those in the reference translations. ROUGE stands for Recall-Oriented Understudy for Gisting Evaluation, which measures the overlap between the n-grams of the system-generated summary and those reference summaries.

For evaluating the diversity of the generated explanations, Unique Sentence Ratio(USR), Feature Coverage Ratio(FCR) and Feature Diversity(FD) can be taken into consideration. [21]

$$USR = |\mathcal{S}|/\mathcal{N} \quad (7)$$

where $\mathcal{S}$ denotes the set of generated unique explanations, and $\mathcal{N}$ is the number of total explanations.

$$FCR = \frac{\mathcal{N}_\mathcal{F}}{|\mathcal{F}|} \quad (8)$$

This metric is to measure how many different features are shown in the generated explanations, $\mathcal{N}_\mathcal{F}$ is the number of distinct features, and $\mathcal{F}$ is the collection of all features in the dataset.

$$FD = \frac{2}{N \times (N - 1)} \sum_{u,w,i,it} |\hat{\mathcal{F}}_{u,i} \cup \hat{\mathcal{F}}_{w,it}| \quad (9)$$

where $\hat{\mathcal{F}}_{u,i}$ and $\hat{\mathcal{F}}_{w,it}$ respectively represent two sets of features contained in two generated explanations. This metric is to compute the overall overlap between feature sets, a lower FD scores implies a higher diversity across explanations.

4 Challenges

4.1 Cold-Start Problem

The cold-start problem includes three cases, new community, new item, and new user[22]. The new community refers to such recommender systems that lack users' interactions, which make them hard to provide reliable recommendations. The new item refers to the situation that a new item is added to the system, but no interactions are present. The new user refers to the situation that a recommender system lacks the new user's interaction, which makes it hard to provide personalized recommendations.

Recently, to solve the cold-start problem, PromptRec[23] considers the user and item profile features as input, transforms them into templated sentences, and predicts recommendations through PLMs. Specifically, they first introduce a verbalizer mapping profile features to natural language descriptions, then apply a template reformatting the recommendation task as Masked Language Modeling(MLM) task, and finally leverage a PLM to accomplish the task and recommendations. Figure 6 shows the structure of PromptRec. It is also possible that applying a similar scheme to Generating explanations for new users.

4.2 Bias Problem

Bias is common in RS[24] for the following components: user behaviors, presented items, feedback loop when training. Firstly, user behavior is observational because user generates behaviors on the basis of the exposed items, making the observational data confounded by the exposure mechanism of the system and self-attention of the user. The presented items are unavoidably biased, some items are more popular than others and thus receive more user behaviors(such as reviews or purchase history). Also, the feedback loop of machine learning can intensify the biases over time.

One of the most significant bias categories in explanation generating is popularity bias(or long-tail problem), which stands for popular items are recommended more frequently while ignoring other items. In particular, for generated explanations, the unfairness issues will occur, influencing the level of personalized recommendation and the trust of the users eventually. UP5[13] intends to solve unfairness problem by designing a personalized prefix prompt that enhances fairness with respect to individual sensitive attributes, which shows an improvement in eliminating unfairness across gender, age, and occupation.

5 Conclusion

In conclusion, the integration of Large Language Models (LLMs) has revolutionized Recommender Systems (RS). This paradigm shift holds tremendous

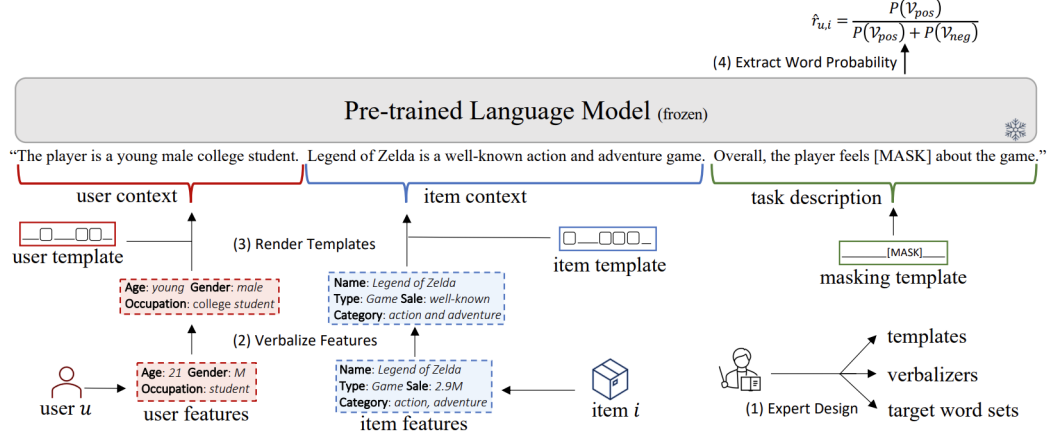


Figure 6: PromptRec: Prompting PLMs to make personalized recommendations under the system cold-start setting

potential for personalized and explainable recommendations, as LLMs offer a versatile toolset for processing vast textual data, thereby enhancing user experiences. Despite these advancements, challenges persist in the realm of Personalized Explanation Generating (PEG) tasks, including cold-start problems and issues of unfairness and bias in RS. Addressing these challenges is crucial for optimizing the potential of LLMs in recommendation systems, ensuring not only accuracy but also transparency and fairness in explaining personalized recommendations to users.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [2] Robert Dale. Gpt-3: What’s it good for? *Natural Language Engineering*, 27(1):113–118, 2021.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [4] Peng Liu, Lemei Zhang, and Jon Atle Gulla. Pre-train, Prompt and Recommendation: A Comprehensive Survey of Language Modelling Paradigm Adaptations in Recommender Systems. *arXiv e-prints*, page arXiv:2302.03735, February 2023.
- [5] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*, pages 299–315, 2022.
- [6] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.
- [7] Peng Liu, Lemei Zhang, and Jon Atle Gulla. Pre-train, prompt and recommendation: A comprehensive survey of language modelling

- paradigm adaptations in recommender systems. *arXiv preprint arXiv:2302.03735*, 2023.
- [8] Ilya Sutskever, James Martens, and Geoffrey E Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 1017–1024, 2011.
 - [9] Hugo Touvron and Louis Martin et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
 - [10] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer, 2019.
 - [11] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. Transformers4rec: Bridging the gap between nlp and sequential/session-based recommendation. In *Proceedings of the 15th ACM Conference on Recommender Systems*, pages 143–153, 2021.
 - [12] Aleksandr V Petrov and Craig Macdonald. Generative sequential recommendation with gptrec. *arXiv preprint arXiv:2306.11114*, 2023.
 - [13] Wenyue Hua, Yingqiang Ge, Shuyuan Xu, Jianchao Ji, and Yongfeng Zhang. Up5: Unbiased foundation model for fairness-aware recommendation. *arXiv preprint arXiv:2305.12090*, 2023.
 - [14] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 585–593, 2022.
 - [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv e-prints*, page arXiv:2106.09685, June 2021.
 - [16] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. A survey on large language models for recommendation. *arXiv preprint arXiv:2305.19860*, 2023.
 - [17] Lei Li, Yongfeng Zhang, and Li Chen. Personalized prompt learning for explainable recommendation. *ACM Transactions on Information Systems*, 41(4):1–26, 2023.
 - [18] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
 - [19] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
 - [20] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.
 - [21] Lei Li, Yongfeng Zhang, and Li Chen. Generate neural template explanations for recommendation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 755–764, 2020.
 - [22] Jesús Bobadilla, Fernando Ortega, Antonio Hernandez, and Jesús Bernal. A collaborative filtering approach to mitigate the new user cold start problem. *Knowledge-based systems*, 26:225–238, 2012.
 - [23] Xuansheng Wu, Huachi Zhou, Wenlin Yao, Xiao Huang, and Ninghao Liu. Towards personalized cold-start recommendation with prompts. *arXiv preprint arXiv:2306.17256*, 2023.
 - [24] Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. Bias and debias in recommender system: A survey and future directions. *ACM Transactions on Information Systems*, 41(3):1–39, 2023.